

LECTURE 5: INTRODUCTION
TO RISC-V &
REGISTER CONVENTIONS

Xv6

- xv6 is a simple OS for easy teaching of OS concepts
 - Two versions, one for x86 hardware and one for RISC-V hardware
 - This series of lectures based on **RISC-V version**
 - <https://github.com/mit-pdos/xv6-public>
- Xv6, provides the basic interfaces introduced by Ken Thompson and Dennis Ritchie's Unix operating system
- It also mimics the Unix's internal design.

Xv6 – RISC V

- Xv6 runs on a multi-core RISC-V microprocessor, and much of its low-level functionality (for example, its process implementation) is specific to RISC-V.
- RISC-V is a 64-bit CPU, and xv6 is written in “LP64” C, which means long (L) and pointers (P) in the C programming language are 64 bits, but an int is 32 bits.
- Note the CPU in a complete computer is surrounded by support hardware, much of it in the form of I/O interfaces.
- Xv6 is written for the support hardware simulated by qemu’s “-machine virt” option. This includes RAM, a ROM containing boot code, a serial connection to the user’s keyboard/screen, and a disk for storage.

Part A: BASICS OF RISC-V

Quick Look at RISC-V

- What is RISC-V?
- A high-quality, license-free, royalty-free RISC ISA
- Standard maintained by the non-profit RISC-V Foundation
- Suitable for all types of computing systems
 - From Microcontrollers to Supercomputers
- RISC-V is available freely under a permissive license
- RISC-V is not...
- A Company
- A CPU implementation

RISC-V Foundation – riscv.org

[ABOUT](#)[SPECIFICATIONS](#)[DEVELOPERS](#)[INDUSTRIES](#)[COMMUNITY](#)[MEMBERS](#)[JOIN](#)

About RISC-V International

RISC-V International is the global non-profit home of the open standard RISC-V Instruction Set Architecture (ISA), related specifications, and stakeholder community.

[JOIN RISC-V INTERNATIONAL](#)[GET IN TOUCH](#)

RISC-V enables the community to share technical investment, contribute to the strategic future, create more rapidly, enjoy unprecedented design freedom, and substantially reduce the cost of innovation.

More than 4,500 RISC-V members across 70 countries contribute and collaborate to define RISC-V open specifications as well as convene and govern related technical, industry, domain, and special interest groups. RISC-V combines a modular technical approach with an open, royalty-free ISA — meaning that anyone, anywhere can benefit from the IP contributed and produced by RISC-V. As a non-profit, RISC-V does not maintain any commercial interest in products or services. As an open standard, anyone may leverage RISC-V as a building block in their open or proprietary solutions and services.

[View Presentation](#)

Register File

- RV32I/64I have 32 Integer Registers
 - Optional 32 FP registers with the F and D extensions
- RV32E reduces the register file to 16 integer registers for area constrained embedded devices
- Width of Registers is determined by ISA
- RISC-V Application Binary Interface (ABI) defines standard functions for registers
 - Allows for software interoperability
- Development tools usually use ABI names for simplicity
- Later we will discuss caller-callee

Register	ABI Name	Description	Saver
x0	zero	Hard-wired zero	-
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	-
x4	tp	Thread pointer	-
x5-7	t0-2	Temporaries	Caller
x8	s0/fp	Saved register/Frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function Arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller

RISC-V Modes

- RISC-V Privileged Specification defines 3 levels of privilege, called Modes
- Machine mode is the highest privileged mode and the only required mode
 - Flexibility allows for a range of targeted implementations from simple MCUs to high-performance Application Processors
- Machine, Hypervisor, Supervisor modes each have Control and Status Registers (CSRs)

RISC-V Modes		
Level	Name	Abbr.
0	User/Application	U
1	Supervisor	S
2	Hypervisor	HS
3	Machine	M

Supported Combinations of Modes	
Supported Levels	Modes
1	M
2	M, U
3	M, S, U
4	M, HS, S, U

Instruction Format

32-bit RISC-V instruction formats																															
Format	Bit																														
	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8							
Store	imm[11:5]							rs2					rs1					funct3			imm[4:0]										
Branch	[12]	imm[10:5]																			imm[4:1]										
Register/ register	funct7																				rd										
Immediate	imm[11:0]																														
Upper immediate	imm[31:12]																														
Jump	[20]	imm[10:1]										[11]	imm[19:12]																		
• opcode (7 bits): Partially specifies one of the 6 types of <i>instruction formats</i>. • funct7 (7 bits) and funct3 (3 bits): These two fields extend the <i>opcode</i> field to specify the operation to be performed. • rs1 (5 bits) and rs2 (5 bits): Specify, by index, the first and second operand registers respectively (i.e., source registers). • rd (5 bits): Specifies, by index, the destination register to which the computation result will be directed.																															

Instructions

Base Integer Instructions: RV32I, RV64I, and RV128I					
Category	Name	Fmt	RV32I Base		+RV{64,128}
Loads	Load Byte	I	LB	rd,rs1,imm	
	Load Halfword	I	LH	rd,rs1,imm	
	Load Word	I	LW	rd,rs1,imm	L{D Q} rd,rs1,imm
	Load Byte Unsigned	I	LBU	rd,rs1,imm	
	Load Half Unsigned	I	LHU	rd,rs1,imm	L{W D}U rd,rs1,imm
Stores	Store Byte	S	SB	rs1,rs2,imm	
	Store Halfword	S	SH	rs1,rs2,imm	
	Store Word	S	SW	rs1,rs2,imm	S{D Q} rs1,rs2,imm
Shifts	Shift Left	R	SLL	rd,rs1,rs2	SLL{W D} rd,rs1,rs2
	Shift Left Immediate	I	SLLI	rd,rs1,shamt	SLLI{W D} rd,rs1,shamt
	Shift Right	R	SRL	rd,rs1,rs2	SRL{W D} rd,rs1,rs2
	Shift Right Immediate	I	SRLI	rd,rs1,shamt	SRLI{W D} rd,rs1,shamt
	Shift Right Arithmetic	R	SRA	rd,rs1,rs2	SRA{W D} rd,rs1,rs2
	Shift Right Arith Imm	I	SRAI	rd,rs1,shamt	SRAI{W D} rd,rs1,shamt
Arithmetic	ADD	R	ADD	rd,rs1,rs2	ADD{W D} rd,rs1,rs2
	ADD Immediate	I	ADDI	rd,rs1,imm	ADDI{W D} rd,rs1,imm
	SUBtract	R	SUB	rd,rs1,rs2	SUB{W D} rd,rs1,rs2
	Load Upper Imm	U	LUI	rd,imm	
	Add Upper Imm to PC	U	AUIPC	rd,imm	

Base Integer Instructions: RV32I, RV				
Category	Name	Fmt	RV32I Base	
Logical	XOR	R	XOR	rd,rs1,rs2
	XOR Immediate	I	XORI	rd,rs1,imm
	OR	R	OR	rd,rs1,rs2
	OR Immediate	I	ORI	rd,rs1,imm
	AND	R	AND	rd,rs1,rs2
	AND Immediate	I	ANDI	rd,rs1,imm
Compare	Set <	R	SLT	rd,rs1,rs2
	Set < Immediate	I	SLTI	rd,rs1,imm
	Set < Unsigned	R	SLTU	rd,rs1,rs2
	Set < Imm Unsigned	I	SLTIU	rd,rs1,imm
Branches	Branch =	SB	BEQ	rs1,rs2,imm
	Branch ≠	SB	BNE	rs1,rs2,imm
	Branch <	SB	BLT	rs1,rs2,imm
	Branch ≥	SB	BGE	rs1,rs2,imm
	Branch < Unsigned	SB	BLTU	rs1,rs2,imm
	Branch ≥ Unsigned	SB	BGEU	rs1,rs2,imm
Jump & Link	J&L	UJ	JAL	rd,imm
	Jump & Link Register	UJ	JALR	rd,rs1,imm
Synch	Synch thread	I	FENCE	
	Synch Instr & Data	I	FENCE.I	
System	System CALL	I	SCALL	
	System BREAK	I	SBREAK	
Counters	Read CYCLE	I	RDCYCLE	rd
	Read CYCLE upper Half	I	RDCYCLEH	rd
	Read TIME	I	RDTIME	rd
	Read TIME upper Half	I	RDTIMEH	rd
	Read INSTR RETired	I	RDINSTRET	rd
	Read INSTR upper Half	I	RDINSTRETH	rd

RISC-V Caller & Callee Saved Registers: Key Terms

- **Caller:** a procedure that calls one or more subsequent procedure(s).
- **Callee:** a procedure that is called by another.
- **Application Binary Interface (ABI):** a standard for register usage and memory layout that allows for programs that are not compiled together to interact effectively.
- **Calling Conventions:** a subset of an ABI specifically focused on how data is passed from one procedure to another.
- Importantly, a procedure may be both a *caller* and a *callee*.

Register Savings

- Register savings is done to prevent overwriting
 - Register can get clobbered during function call
 - Two methods
 - Caller saved : registers saved on stack by caller before invoking the function (caller save registers). Function code (callee) can freely change them, caller restores them later on.
 - Callee saved: Some registers saved by callee function and restored after function ends (callee save registers). Caller expects them to have same value on return.
- All of this is automatically done by C compiler (C calling convention)

Function calls

- Format:
 - jal dst, label – Jump to 'label', store PC+4 in dst
 - jalr dst, offset(base) – Jump to rf[base]+offset, store PC+4 in dst
 - Eg. jalr x1, 4(x5) Jumps to x5+4, stores PC+4 in x1
- Why do we need two variants?
 - jal has a limit on how far it can jump
 - (Why? Encoding issues explained later)
- jalr used to jump to locations defined at runtime
 - Needed for many things including function calls (e.g., Many callers calling one function)

```
...  
jal x1, function1  
...  
function1:  
...  
jalr x0, 0(x1)
```

Calling conventions and stack

- **ra (x1)**: typically holding return address
 - Saver is “caller”, meaning a function caller must save its **ra** somewhere before calling
- **sp (x2)**: typically used as stack pointer
- **t0-t6**: temporary registers
 - Saver is “caller”, meaning a function caller must save its values somewhere before calling, if its values are important (Callee can use it without worrying about losing value)
- **a0-a7**: arguments to function calls and return value
 - Saver is “caller”
- **s0-s11**: saved register
 - Saver is “callee”, meaning if a function wants to use one, it must first save it somewhere, and restore it before returning

Calling conventions and stack

- Registers saved in off-chip memory across function calls
- Stack pointer **x2 (sp)** used to point to top of stack
 - **sp** is callee-save
 - No need to save if callee won't call another function
- Stack space is allocated by decreasing value
 - Referencing done in sp-relative way
- Aside: Dynamic data used by heap for malloc

Example: Using callee-saved registers

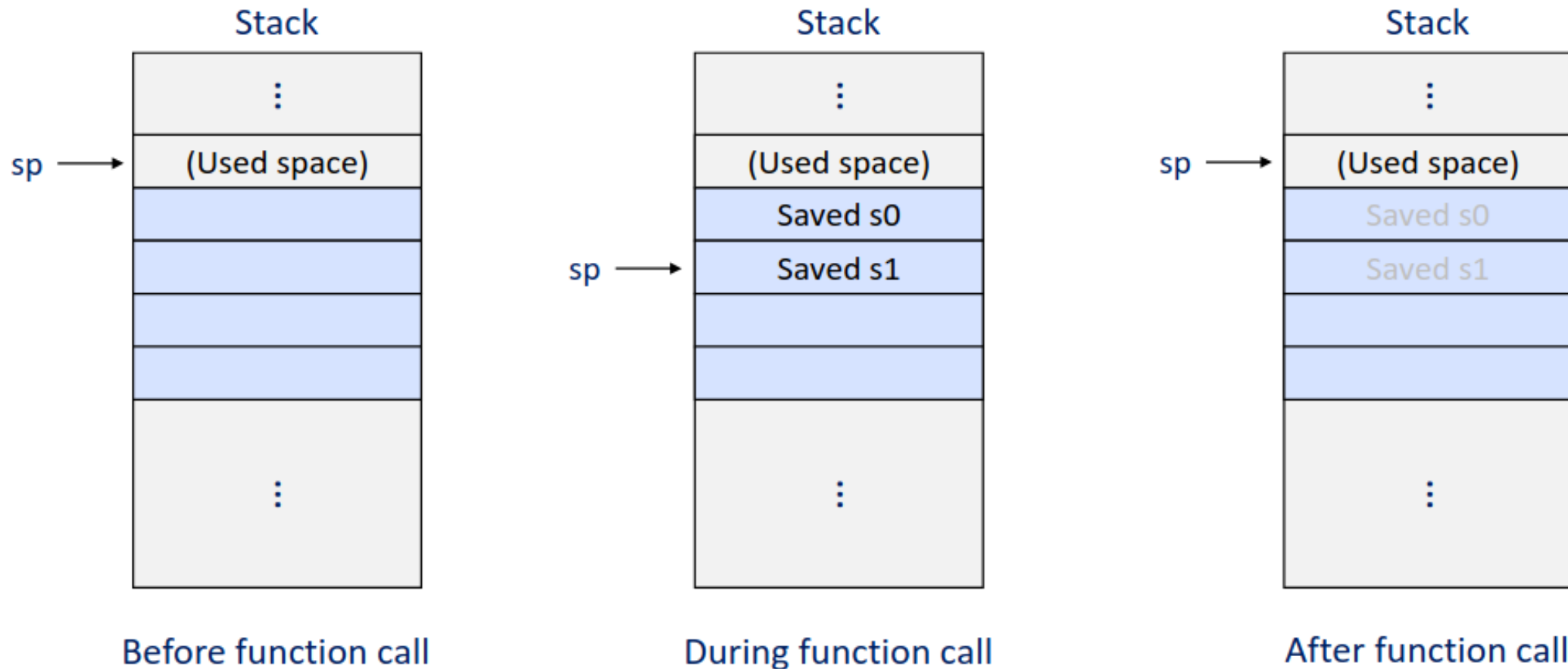
❑ Will use s0 and s1 to implement f

```
int f(int x, int y) {  
    return (x + 3) | (y + 123456);  
}
```

f:

```
addi sp, sp, -8    // allocate 2 words (8 bytes) on stack  
sw s0, 4(sp)       // save s0  
sw s1, 0(sp)       // save s1  
addi s0, a0, 3  
li s1, 123456  
add s1, a1, s1  
or a0, s0, s1  
lw s1, 0(sp)       // restore s1  
lw s0, 4(sp)       // restore s0  
addi sp, sp, 8     // deallocate 2 words from stack  
                    // (restore sp)  
ret
```


Example: Using callee-saved registers



Example: Using caller-saved registers

Caller

```
int x = 1;
int y = 2;
int z = sum(x, y);
int w = sum(z, y);
```



```
li a0, 1
li a1, 2
addi sp, sp, -8
sw ra, 0(sp)
sw a1, 4(sp) // save y
jal ra, sum
// a0 = sum(x, y)
lw a1, 4(sp) // restore y
jal ra, sum
// a0 = sum(z, y)
lw ra, 0(sp)
addi sp, sp, 8
```

Callee

```
int sum(int a, int b) {
    return a + b;
}
```



```
sum:
    add a0, a0, a1
    ret
```

ra is saved, meaning even if callee calls another function, caller can still retrieve its ra

Why did the caller save s1?
We don't know which registers callee will use
Caller must save all caller-save registers it cares about

Rule of thumb for register conventions

- ❑ Assume function “foo” calls function “bar”
- ❑ There are two sets of general purpose registers, t’s (t0-t6) and s’s (s0-s11)
 - Saved registers (s’s) are callee-save, meaning “bar” must store them somewhere if it wants to use some
 - Temporary registers (t’s) are caller-save, meaning “foo” must save them somewhere if it wants their values to be the same after returning from “bar”
- ❑ Argument registers (a’s) are caller-save
 - If “bar” wants to call another function “bar2”, it must save the a’s it was given, before setting them to its own arguments (which is natural)

Rule of thumb for register conventions

❑ Rule of thumb for saved registers

- For computation ongoing across function (“bar”) calls, use s’s
- *Simple to just use s’s for most register usage*
- Each function (“bar”) stores all s’s (it plans to use) in the stack at beginning, and restore them before returning

❑ Rule of thumb for temporary registers

- Use t’s for intermediate values that are no longer important after the function call, for example calculating arguments for “bar”.
- “Foo” must store t’s in stack (if it wants their values to persist) before calling “bar”, but *simpler to just restrict use of t’s for values we don’t expect to persist*

Rule of thumb for register conventions

□ Only use callee-save registers for computation (s's)

- At beginning of function: store ra and all s's it will use
- At end of function: restore ra and all s's from stack
- Of course, a's must be handled accordingly (caller-save)
 - Before "foo" calls "bar", "foo" stores all a's in stack
 - After "bar" returns, restores all a's from stack (after copying return value from a0, etc)



Thank You